Thin-Client Interactive Gaussian Adaptive Streaming over HTTP/3

Anonymous Author(s)

Abstract

Recent advancements in 3D Gaussian Splatting (3DGS) have enabled photorealistic rendering of complex scenes, yet widespread adoption on mobile and Extended Reality (XR) devices is hindered by substantial computational and bandwidth requirements. While existing solutions often focus on model compression for client-side rendering, they still demand significant GPU power, limiting their applicability on resource-constrained hardware.

In this paper, we propose TIGAS (Thin-client Interactive Gaussian Adaptive Streaming), a remote rendering framework that offloads the heavy lifting of rasterization to the backend. TIGAS streams view-dependent 2D projections to a lightweight web client over QUIC, to minimize head-of-line blocking and transport latency. To ensure smooth 6DoF navigation under fluctuating network conditions, we introduce a receiver-driven ABR algorithm that adjusts the rendering quality to maintain motion-to-photon latency within levels required for responsive 6DoF interaction. Experimental results using the "Western Pacific Train" dataset demonstrate that TIGAS achieves real-time streaming of 3DGS, with the vast majority of frames being sent at the highest quality level, resulting in a VMAF of up to 98.

The source code will be released upon acceptance of this work.

CCS Concepts

• **Computing methodologies** → **Rasterization**; **Graphics input devices**; *Mixed / augmented reality*; **Point-based models**; *Volumetric models*; • **Networks** → *Network performance evaluation.*

Keywords

3D Gaussian Splatting, Remote Rendering, HTTP/3, QUIC, HTTP Adaptive Streaming, 6DoF Navigation

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

ACM Reference Format:
Anonymous Author(s). 2026. Thin-Client Interactive Gaussian Adaptive Streaming over HTTP/3. In . ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

The proliferation of commercial Extended Reality (XR) devices, such as the Apple Vision Pro and Meta Quest 3, has accelerated the demand for immersive, photo-realistic 3D content. Unlike traditional 2D video, these applications require six Degrees of Freedom (6DoF) navigation, allowing users to explore virtual environments from arbitrary viewpoints. To meet the visual fidelity expectations of modern users, 3D Gaussian Splatting (3DGS) [15] has emerged as a state-of-the-art rendering technique. By representing scenes as millions of anisotropic 3D Gaussians, 3DGS achieves photorealism comparable to Neural Radiance Fields (NeRFs) [21] but with significantly faster real-time rendering speeds [8]. However, widespread adoption of 3DGS on mobile and commodity hardware faces a "delivery gap." High-fidelity 3DGS models are data-intensive, often requiring gigabytes of storage for complex scenes, which translates to prohibitive bandwidth requirements for real-time streaming [32]. Furthermore, while 3DGS rendering is faster than NeRFs, it remains computationally expensive. Rendering millions of splats imposes a heavy load on the GPU, often exceeding the thermal and power budgets of mobile devices. Recent benchmarks indicate that even capable edge platforms, such as the NVIDIA Jetson AGX Orin, struggle to maintain consistent 90 fps seamless motion-sickness-free immersion in complex interactive scenes [29]. Existing 3DGS streaming solutions largely focus on *native streaming*, where raw Gaussian data (or a compressed version thereof) is transmitted to the client for local rendering [24, 25]. While effective for powerful desktops, this approach excludes resource-constrained "thin" clients (e.g., smartphones, smart TVs, basic laptops) that lack the dedicated GPU hardware required for high-frequency rasterization. In this paper, we propose **TIGAS** (Thin-client Interactive Gaussian Adaptive Streaming), a remote rendering framework that decouples visual quality from client-side hardware capabilities. Instead of streaming the scene geometry, TIGAS offloads the heavy lifting of rasterization to a GPU-accelerated backend. The backend generates view-dependent 2D projections based on the client's 6DoF pose and streams them as a sequence of images. To ensure a smooth interactive experience, TIGAS addresses two critical challenges

in remote rendering: *(i)* latency and *(ii)* network instability. First, it leverages QUIC [5] as the transport protocol. Unlike TCP, which is often used in traditional DASH streaming via HTTP/1.1 [10] or HTTP/2 [3], QUIC [13] mitigates Head-of-Line (HoL) blocking, ensuring that a lost packet pertaining to one frame does not stall the delivery of subsequent frames, a crucial feature for minimizing motion-to-photon (MTP) latency [30]. Second, TIGAS introduces a receiver-driven Adaptive Bitrate (ABR) algorithm. Rather than buffering future frames (causing input lag), TIGAS's ABR dynamically adjusts the rendering resolution and compression quality to match the estimated network throughput, targeting a strict interaction latency threshold of 100 ms. Experimental results using the "Western Pacific Train" dataset [15] demonstrate that TIGAS enables real-time, interactive streaming of 3DGS models, at quality up to 98 VMAF points.

2 Related Work

The rapid adoption of 3DGS has encouraged researchers to optimize its storage, transmission, and rendering efficiency. We categorize existing approaches into model compression, native streaming, and remote rendering.

2.1 3DGS Compression and Simplification

A standard 3DGS scene represents the 3D environment as a collection of anisotropic Gaussian primitives. Each primitive is defined by a set of geometric and appearance attributes: a center coordinate, a scale vector, and a rotation angle. Visual appearance is governed by an opacity value and a set of Spherical Harmonics (SH) coefficients that model view-dependent color and specular effects [15]. To reduce the memory footprint, recent work has applied quantization and pruning techniques to 3DGS primitives. [7] proposed to prune Gaussians based on their contribution to the final image, significantly reducing model size without major visual degradation. Similarly, [11] and [22] explored vector quantization and entropy coding to compress Gaussian attributes. While these methods reduce storage requirements, they do not alleviate the rendering bottleneck on the client device; the client must still decompress and rasterize the full set of primitives.

2.2 Native 3DGS Streaming

Native streaming architectures transmit the 3D model data to the client, often employing Level of Detail (LoD) or spatial partitioning to manage bandwidth. [25] introduced LTS, a DASH-based framework that organizes Gaussians into layers (quality), tiles (space), and segments (time). LTS enables clients to request only the necessary subset of data based on their viewport. Similarly, LapisGS [24] utilized a layered approach with progressive rendering, allowing clients to download a coarse base layer followed by refinement layers. Certain frameworks, like GenStream [2] attempt to resolve the 3DGS bandwidth bottleneck by pre-streaming the static environment model and transmitting only the essential attributes of dynamic scene actors in real-time, to be overlaid on the 3DGS model. Other works focus on the integration of 3DGS with standard video codecs. [19] and [12] proposed encoding dynamic Gaussian attributes into video frames to leverage hardware video decoders. [31] explored voxel-based partitioning to enable view-frustum culling during transmission. Despite these optimizations, native streaming inherently relies on the client's GPU for rasterization. As noted by [29], achieving high frame rates on edge devices often requires hardware-algorithm co-design, which is not feasible for generic web clients.

2.3 Interactive Media Remote Rendering

Remote rendering shifts the computational burden to the backend, streaming rendered 2D images or videos to the client. This approach is well-established in Cloud Gaming and Virtual Desktop Infrastructure (VDI) [23]. However, standard video streaming protocols such as HLS/DASH are often ill-suited for the ultra-low latency required by 6DoF interaction [26]. Furthermore, while real-time solutions like WebRTC provide low latency, they often struggle with the flexible ABR logic and rapid state synchronization required for adaptive Gaussian streaming [25]. To address these limitations, recent architectures have turned to QUIC [13] to: *(i)* eliminate HoL blocking for frequently synchronizing 6DoF updates and rendered images, and *(ii)* adopt built-in congestion control and rapid connection establishment (0-RTT) for fast image responses compared to traditional HTTP/1.1 or HTTP/2 stacks. In this paper, we shift the computational burden entirely to a remote GPU backend. By introducing a receiver-driven ABR algorithm that enforces an interaction latency threshold, TIGAS decouples visual quality from the hardware constraints of client devices. This shift from geometry-based transmission to adaptive, view-dependent 2D projection streaming provides a robust solution for responsive 6DoF navigation on commodity hardware.

3 TIGAS System Design

Figure 1 shows the architecture of TIGAS comprising three logical components: *(i)* a *GPU Backend* with a *Rendering Engine*, *(ii)* a low-latency *HTTP/3 (QUIC) Protocol Stack* for transmission, and *(iii)* an *Adaptive Client*.

3.1 GPU Backend

PLY Model and Primitives. TIGAS represents the 3D environment using 3DGS, where the scene is stored as a collection of anisotropic Gaussian primitives within a Polygon (PLY) model. Upon initialization, the remote node parses the PLY file and transfers these tensors to the GPU, enabling the rendering engine to project these millions of primitives onto a 2D image plane based on the client's 6DoF pose.

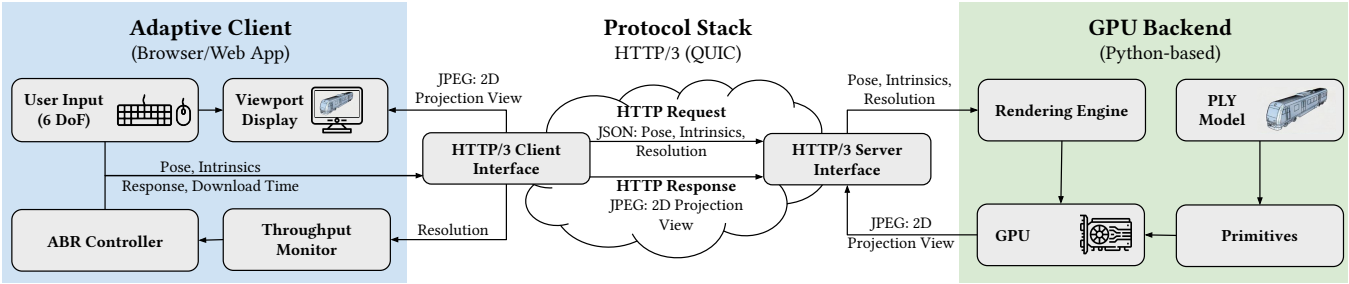

Figure 1: TIGAS consists of an Adaptive Client that handles 6DoF user input and ABR logic, connected via HTTP/3 (QUIC) to a GPU backend performing rasterization, thus offloading rendering from the client.

Rendering Engine. The Rendering Engine serves as the core computational component of the backend, responsible for synthesizing 2D viewpoints from a 3DGS scene. To meet the demands of real-time 6DoF interaction, the engine must leverage a high-performance, tile-based rasterizer that projects 3D Gaussian primitives (position, rotation, scale, opacity, and color) onto a 2D image plane. Furthermore, an ABR algorithm should be implemented to detect a decrease in available bandwidth, and request on-the-fly downsampling of the 2D image, to reduce the transmission payload.

GPU. To minimize the MTP latency essential for interactive XR, the backend is designed to maintain persistent GPU residency for scene data. Upon initialization, Gaussian primitives are pre-loaded into VRAM. This eliminates the I/O overhead associated with per-request model loading.

3.2 Protocol Stack

To achieve low-latency interactive streaming, the communication protocol must support rapid request-response cycles with minimal overhead. We adopt an HTTP/3 (QUIC) architecture to leverage its multiplexing capabilities and resilience against network jitter.

Request-Response Schema. The system is designed around a stateless, per-frame request model. The client transmits a compact JSON payload containing the 6DoF pose, camera intrinsics, and the desired resolution ladder level determined by the ABR controller. The backend treats each request as a separate rendering task, returning a single 2D projection view, encoded via JPEG. This design ensures that user interactions *e.g.*, rapid camera rotation, are immediately reflected in the next network flight without being stalled by previous data.

3.3 Adaptive Client

The client is designed to be "thin", requiring only a standard image decoder and network capabilities, making it compatible with commodity browsers and mobile hardware.

User Input. The client decouples the rendering loop from the input loop. It continuously polls user input to construct a pose request, ensuring that the latest view is always requested regardless of network latency. This decoupling minimizes MTP latency, ensuring that the visual feedback aligns as closely as possible with the user's proprioception, *i.e.*, the ability to sense one's own movement and position, which is critical for mitigating cybersickness in interactive immersive environments.

ABR Controller. To handle network fluctuations without freezing the display, the client implements a receiver-driven ABR algorithm, which selects the optimal combination of rendering resolution and compression quality for each individual frame, to ensure that the rendering and transmission time remains within the interaction threshold target of around 100 ms, corresponding to the latency threshold that humans consider instantaneous [17].

4 Implementation

We implement TIGAS as a client-backend system consisting of a Python-based rendering service and a browser-based frontend.

4.1 GPU Backend

PLY Model and Primitives. The 3DGS scene is ingested from a PLY file containing N Gaussians. The file is parsed into persistent pytorch [16] tensors to eliminate per-request loading overhead and bound latency to pure GPU rasterization plus JPEG encoding. The primitives for each Gaussian n are processed as follows:

- **Spatial Location.** The center of each Gaussian $\boldsymbol{\mu}_n \in \mathbb{R}^3$ is read directly from the PLY vertex properties x, y, z. These are stacked into a means tensor $\mathbf{M} \in \mathbb{R}^{N\times 3}$.
- **Scale.** To enforce positivity, scales are stored as log-vectors $\mathbf{s}_n \in \mathbb{R}^3$. We compute the actual 3D extents by exponentiating them element-wise, $\exp(\mathbf{s}_n)$, and store the result in a scales tensor $\mathbf{S} \in \mathbb{R}^{N\times 3}$.
- **Rotation.** Orientation is parameterized by unit quaternions $\mathbf{q}_n \in \mathbb{R}^4$. These are loaded directly into a quaternion

tensor $\mathbf{Q} \in \mathbb{R}^{N\times 4}$ and normalized if necessary to ensure unit rotations.

- **Opacity.** Attributes stored in logit space are transformed via a sigmoid function $\sigma(x)$ to the range $[0, 1]$ and stored in $\mathbf{O} \in \mathbb{R}^N$.
- **Color and SH.** SH coefficients are reshaped into a tensor $\mathbf{C} \in \mathbb{R}^{N\times 16\times 3}$. Our implementation uses the degree parameter d_{SH} to select the active frequency bands. To minimize memory bandwidth in our baseline streaming scenario, we default to $d_{\text{SH}} = 0$ (view-independent color), while supporting up to degree 3 for high-fidelity specular effects when resources permit.

Rendering Engine. To render the scene, we must express world points in camera coordinates. A *world coordinate space* is a fixed reference frame in which all Gaussians are defined. A *camera coordinate space* is attached to the virtual camera controlled by the user: its origin is at the camera optical center, the z-axis points forward, and the x and y axes span the image plane. The backend uses 6DoF inputs to convert world coordinates into camera coordinates. The backend receives two angles: *azimuth* θ (rotation around the vertical axis) and *elevation* ϕ (rotation around the horizontal axis). Following standard viewing conventions for interactive remote rendering [6], the third angle *i.e.*, roll, rotation around the optical axis, is set to zero. With this information, the backend is able to construct a rotation matrix $\mathbf{R} \in \mathbb{R}^{3\times 3}$. The world-to-camera transformation matrix $\mathbf{T}_{w\to c}$ is then computed as:

$$\mathbf{T}_{w\to c} = \begin{bmatrix} \mathbf{R}^\top & -\mathbf{R}^\top \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \in \mathbb{R}^{4\times 4}.$$

This matrix maps world-space Gaussian centers to camera-space coordinates via: $\mathbf{p}_c = [x_c, y_c, z_c]^\top$. The intrinsics matrix:

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}.$$

accounts for focal lengths (f_x, f_y) and the principal point (c_x, c_y), allowing to project these points to pixel coordinates (u, v) via:

$$\lambda \begin{bmatrix} u & v & 1 \end{bmatrix}^\top = \mathbf{K}\mathbf{p}_c, \quad \text{where } \lambda = z_c.$$

Increasing f_x or f_y magnifies the image (narrower field-of-view). Setting $c_x = W/2$, $c_y = H/2$ centers the projection in an image of width W and height H. After rasterization, we obtain an RGB image $\mathbf{I} \in [0, 1]^{H\times W\times 3}$ as floating-point values. We clamp (to guard against minor numeric overflow), scale to $[0, 255]$, cast to 8-bit integers, and compress with JPEG. The response header forbids caching to ensure each user interaction triggers a fresh rendered 2D image.

4.2 Protocol Stack: HTTP/3 over QUIC

Protocol Framework. TIGAS runs the Asynchronous Gateway Interface (ASGI) app on an `aioquic`-based backend via a `QuicConfiguration` with the appropriate Application-Layer Protocol Negotiation (ALPN) protocols [5]; it sets the congestion control (default `reno`), and enables datagrams up to 64 KB [1].

Server Interface. The backend implements a thin ASGI bridge: incoming QUIC events (headers, data) are mapped to ASGI messages for HTTP/3, WebSocket [9], or WebTransport [28]. To prevent the ASGI loop from stalling during rasterization/JPEG encoding, the synchronous render path is offloaded to a thread via `asyncio.to_thread`, improving interaction responsiveness under load. The response is returned with media type JPEG and header `Cache-Control: no-store` so browsers do not reuse stale images. For local testing over HTTP/1.1, TIGAS includes a minimal Flask backend exposing the same `/render` endpoint. It runs the same tensor preparation and rasterization pipeline.

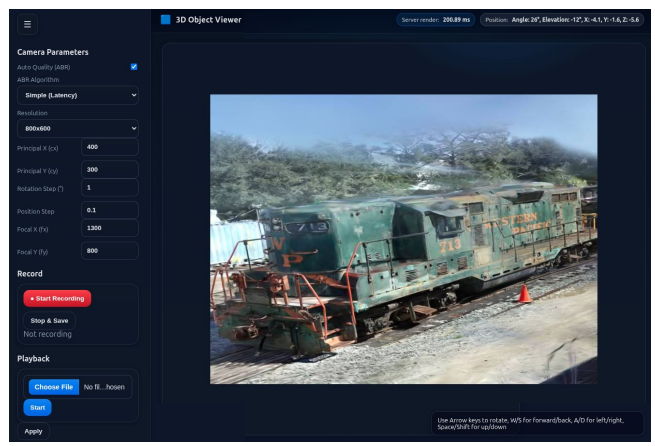

Figure 2: Browser-based client UI showing the viewport and collapsible sidebar for parameter control.

Client Interface. The client-side implements a small web application using the ASGI framework Starlette [20]. The app mounts: (i) a GET route at `/` which serves the single-page (HTML) client, (ii) a POST route at `/render` which returns a freshly rendered JPEG, and (iii) a static files mount at `/static` for JS/CSS and the 3DGS model. The client issues `POST /render` with a JSON body containing camera pose, intrinsics, and target resolution, which is: $\{\theta, \phi, t_x, t_y, t_z, f_x, f_y, c_x, c_y, W, H\}$.

4.3 Adaptive Client

Viewport Display. The HTML client, shown in Figure 2, offers a collapsible sidebar with fields for f_x, f_y, c_x, c_y, resolution, and step sizes. On each update, the client assembles JSON

Figure 3: Visual comparison of rendered frames at different quality levels: (a) high, (b) medium-high, (c) medium-low, and (d) low.

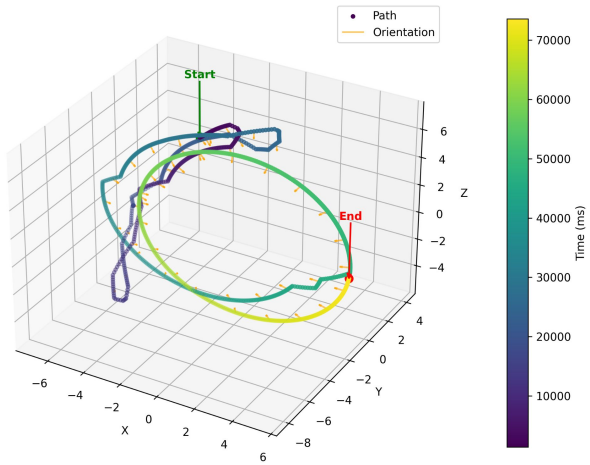

Figure 4: Circular motion around the 3DGS model to fairly compare different ABR/Trace combinations.

with pose, intrinsics, and resolution, then `fetches /render`. The response body, i.e., a JPEG image, is then in the viewport. Eventual errors replace the viewport with a short message; URLs are revoked after use to avoid leaks. When resolution changes, c_x and c_y are set to $W/2$ and $H/2$ to keep the principal point centered; focal lengths are left unchanged for simplicity (in pixels), which effectively alters the field-of-view (FoV). This is deemed acceptable for our prototype. In a real live-streaming environment, the FoV should be kept fixed by scaling f_x, f_y proportionally to resolution.

User Input. Keyboard controls update pose: arrows adjust azimuth/elevation; `W/S` move forward/backward; `A/D` move sideways; `Shift/Space` move up/down. Motion in the x-z plane is computed relative to azimuth θ using $\sin\theta$ and $\cos\theta$ so "forward" follows the camera's current facing. An animation loop polls keys and triggers updates without blocking the UI.

Throughput Monitoring. For each request i, the client records start/end times and reads the `Content-Length` header to estimate the image size S_i in bytes. Instantaneous throughput is computed as $\hat{r}_i = \frac{S_i}{\Delta t_i}$, where Δt_i is the download duration in seconds for request i. To reduce jitter, TIGAS maintains an Exponential Moving Average (EMA) over the last few samples (default history length is 5 requests) with a default smoothing factor $\alpha = 0.3$.

ABR Controller. To manage network fluctuations without interrupting the 6DoF experience, we propose a receiver-driven ABR algorithm, hereafter referred to as the *Latency* ABR. Our algorithm utilizes a predefined bitrate ladder (see Table 1) that maps each quality level ℓ to a specific JPEG compression level and its corresponding expected size $\tilde{S}_\ell$. For any candidate level, the estimated completion time $\tilde{T}_\ell$ (in seconds) is calculated based on the current throughput r_i:

$$\tilde{T}_\ell = \frac{\tilde{S}_\ell}{r_i \cdot 1024}$$

The *Latency* algorithm uses this estimation to target a maximum per-request time of $T_{\text{target}} = 100$ ms, which represents the threshold for instantaneous human perception. To maintain stability during 6DoF navigation, the ABR policy adheres to the following mathematical constraints:

- *Upgrade Condition:* A switch to a higher resolution level $\ell+1$ occurs only if the predicted time satisfies $\tilde{T}_{\ell+1} \leq T_{\text{target}}$.
- *Downgrade Condition:* To mitigate the impact of network jitter, a downgrade to level $\ell - 1$ is triggered if the current completion time consistently exceeds the safety margin, defined as $\tilde{T}_\ell > T_{\text{margin}}$.
- *Safety Constraint:* We maintain the relationship $T_{\text{target}} < T_{\text{margin}}$ to create a deadband that prevents frequent oscillations.

Table 1: Frame Quality Levels (Bitrate Ladder)

Quality (Profile)	JPEG Compression (%)	Size (KB)
Low (3)	10	2–3
Medium–Low (2)	35	6–8
Medium–High (1)	65	18–22
High (0)	90	50–60

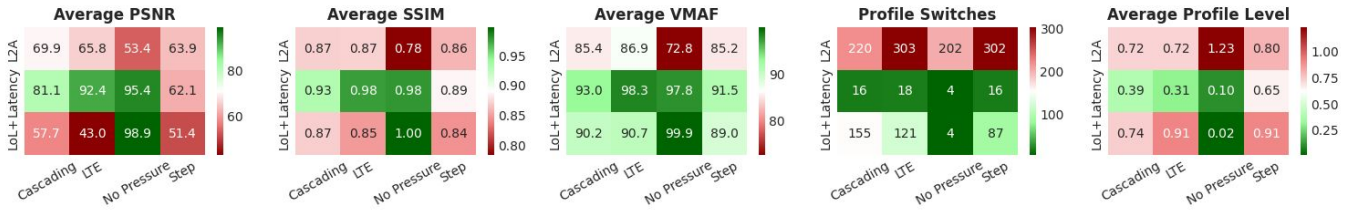

Figure 5: Performance summary for each combination of ABR algorithm and network trace.

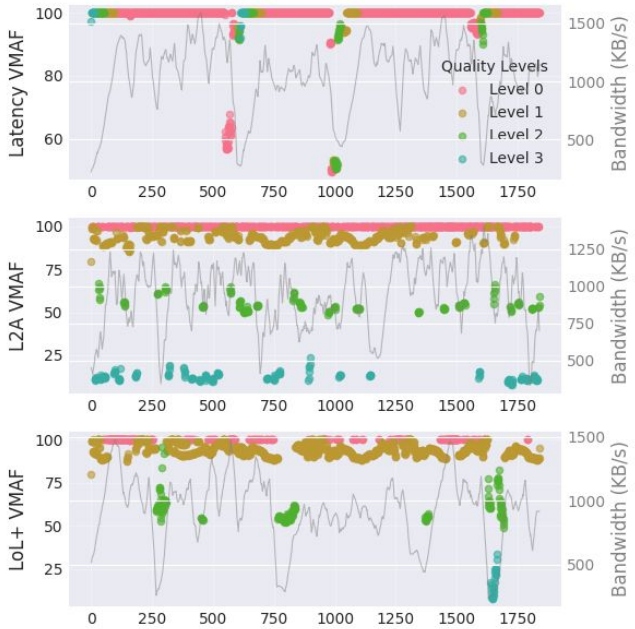

Figure 6: Frame-level VMAF profile distribution for the LTE trace.

To further enhance stability, a hold counter requires $h = 3$ consecutive requests before a resolution change is finalized. When ABR is active, the manual resolution UI is disabled to ensure consistent performance. Following any resolution switch, TIGAS automatically resets the principal point to $c_x = W/2$ and $c_y = H/2$ to keep the 2D projection centered. In instances where the `Content-Length` header is missing, the system defaults to a conservative estimate to favor lower-latency resolutions.

5 Performance Evaluation

Setup. We evaluate TIGAS using a backend equipped with dual NVIDIA Quadro RTX 8000 GPUs and a commodity laptop client (Intel Iris Xe). We use the "Western Pacific Train" model [15] and the four-level bitrate ladder detailed in Table 1 and Figure 3. To ensure reproducible benchmarking across different conditions, we simulate client input using a fixed, pre-recorded circular trajectory (Figure 4) rather than live manual interaction. We compare TIGAS's Latency ABR against SOTA algorithms (LOL+ [18], L2A [14]) adapted for frame-level, requests against four standardized network traces (Cascading, LTE, Step, No Pressure *i.e.*, no bandwidth throttling) [4, 27]. The backend self-throttles, by calculating the expected transmission and propagation delay for each response according to the active network trace and delaying packet dispatch accordingly. Each combination was run once, for the whole 73 seconds duration of Figure 4's circular motion, resulting in around 1800 frames per experiment.

Results. Figure 5 summarizes performance with respect to PSNR, SSIM, VMAF, profile switches, and per-frame average quality level in Table 1. The results identify the *Latency* algorithm as the most robust performer, effectively balancing visual fidelity with interaction latency. In contrast, *LOL+* exhibits aggressive switching in unthrottled scenarios, while *L2A* underperforms significantly, possibly due to suboptimal translation from segment-based logic. Frame-level analysis (Figure 6) confirms the previous results. Under the example LTE conditions, TIGAS predominantly maintains high-quality transmission, with Latency ABR rarely needing to drop from the high profile quality.

6 Conclusions

In this work, we presented TIGAS, a thin-client framework for interactive 3DGS streaming over HTTP/3. By offloading rasterization to the backend and employing frame-level ABRs, TIGAS decouples visual fidelity from client hardware, enabling real-time, 6DoF navigation on commodity devices, with quality up to 98 VMAF points. While our proof-of-concept validates the architecture using the "Western Pacific Train" dataset, future work will focus on diverse scene benchmarks and native frame-level ABR optimization. Although shifting computation to the cloud increases backend costs, this architectural trade-off is currently vital for overcoming the GPU limitations of mobile hardware and democratizing access to immersive 3D content.

References

[1] Mark Allman, Ethan Blanton, and Vern Paxson. 2009. TCP Congestion Control. RFC 5681. doi:10.17487/RFC5681

[2] Emanuele Artioli, Daniele Lorenzi, Shivi Vats, Farzad Tashtarian, and Christian Timmerer. 2025. GenStream: Semantic Streaming Framework for Generative Reconstruction of Human-centric Media. In *Proceedings of the 33rd ACM International Conference on Multimedia* (Dublin, Ireland) *(MM '25)*. Association for Computing Machinery, New York, NY, USA, 12276–12284. doi:10.1145/3746027.3758153

[3] Mike Belshe, Roberto Peon, and Martin Thomson. 2015. Hypertext Transfer Protocol Version 2 (HTTP/2). https://httpwg.org/specs/rfc7540.html. doi:10.17487/RFC7540

[4] Abdelhak Bentaleb et al. 2020. ACM MMSys 2020 Grand Challenge: Twitch Low-Latency Live Streaming Challenge. In *Proceedings of the 11th ACM Multimedia Systems Conference*.

[5] Mike Bishop. 2022. HTTP/3. RFC 9114. doi:10.17487/RFC9114

[6] James Diebel et al. 2006. Representing attitude: Euler angles, unit quaternions, and rotation vectors. *Matrix* 58, 15-16 (2006), 1–35.

[7] Zhiwen Fan, Kevin Wang, Kairun Wen, Zehao Zhu, Dejia Xu, and Zhangyang Wang. 2023. LightGaussian: Unbounded 3D Gaussian Compression with 15x Reduction and 200+ FPS. arXiv:2311.17245 [cs.CV]

[8] Ben Fei, Jingyi Xu, Rui Zhang, Qingyuan Zhou, Weidong Yang, and Ying He. 2025. 3D Gaussian Splatting as a New Era: A Survey. *IEEE Transactions on Visualization and Computer Graphics* 31, 8 (2025), 4429–4449.

[9] Ian Fette and Alexey Melnikov. 2011. The WebSocket Protocol. RFC 6455. doi:10.17487/RFC6455

[10] Roy T. Fielding, Jim Gettys, Jeffrey Mogul, Henrik Frystyk, Larry Masinter, Paul Leach, and Tim Berners-Lee. 1999. Hypertext Transfer Protocol – HTTP/1.1. https://tools.ietf.org/html/rfc2616. doi:10.17487/RFC2616

[11] Sharath Girish, Kamal Gupta, and Abhinav Shrivastava. 2024. EAGLES: Efficient Accelerated 3D Gaussians with Lightweight EncodingS. arXiv:2312.04564 [cs.CV] https://arxiv.org/abs/2312.04564

[12] Qiang Hu, Zihan Zheng, Houqiang Zhong, Sihua Fu, Li Song, Xiaoyun Zhang, Guangtao Zhai, and Yanfeng Wang. 2025. 4DGC: Rate-Aware 4D Gaussian Compression for Efficient Streamable Free-Viewpoint Video. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 875–885.

[13] Jana Iyengar and Martin Thomson. 2021. QUIC: A UDP-Based Multiplexed and Secure Transport. https://www.rfc-editor.org/rfc/rfc9000.html. doi:10.17487/RFC9000

[14] Theo Karagkioules, Rufael Mekuria, Dirk Griffioen, and Arjen Wagenaar. 2020. Online learning for low-latency adaptive streaming. In *Proceedings of the 11th ACM Multimedia Systems Conference* (Istanbul, Turkey) *(MMSys '20)*. Association for Computing Machinery, New York, NY, USA, 315–320. doi:10.1145/3339825.3397042

[15] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkuehler, and George Drettakis. 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics* 42, 4 (July 2023). doi:10.1145/3592433

[16] Nikhil Ketkar, Jojo Moolayil, Nikhil Ketkar, and Jojo Moolayil. 2021. *Deep Learning with Python: Learn Best Practices of Deep Learning Models with PyTorch*. Springer.

[17] Tanja Kojic, Steven Schmidt, Sebastian Möller, and Jan-Niklas Voigt-Antons. 2019. Influence of Network Delay in Virtual Reality Multiplayer Exergames: Who is actually delayed?. In *2019 Eleventh International Conference on Quality of Multimedia Experience (QoMEX)*. 1–3. doi:10.1109/QoMEX.2019.8743342

[18] May Lim, Mehmet N. Akcay, Abdelhak Bentaleb, Ali C. Begen, and Roger Zimmermann. 2020. When they go high, we go low: low-latency live streaming in dash.js with LoL. In *Proceedings of the 11th ACM Multimedia Systems Conference* (Istanbul, Turkey) *(MMSys '20)*. Association for Computing Machinery, New York, NY, USA, 321–326. doi:10.1145/3339825.3397043

[19] Bangya Liu and Suman Banerjee. 2024. Swings: Sliding window Gaussian splatting for volumetric video streaming with arbitrary length. *arXiv preprint arXiv:2409.07759* (2024).

[20] Encode OSS Ltd. 2024. *Starlette: The little ASGI framework that shines*. https://www.starlette.io/

[21] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. 2020. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. In *European Conference on Computer Vision (ECCV)*.

[22] KL Navaneet, Kossar Pourahmadi Meibodi, Soroush Abbasi Koohpayegani, and Hamed Pirsiavash. 2024. CompGS: Smaller and Faster Gaussian Splatting with Vector Quantization. arXiv:2311.18159 [cs.CV] https://arxiv.org/abs/2311.18159

[23] Shu Shi and Cheng-Hsin Hsu. 2015. A survey of interactive remote rendering systems. *ACM Computing Surveys (CSUR)* 47, 4 (2015), 1–29.

[24] Yuang Shi, Géraldine Morin, Simone Gasparini, and Wei Tsang Ooi. 2025. LapisGS: Layered Progressive 3D Gaussian Splatting for Adaptive Streaming. In *Proceedings of the 2025 International Conference on 3D Vision (3DV)*. IEEE, 991–1000.

[25] Yuan-Chun Sun, Yuang Shi, Cheng-Tse Lee, Mufeng Zhu, Wei Tsang Ooi, Yao Liu, Chun-Ying Huang, and Cheng-Hsin Hsu. 2025. LTS: A DASH Streaming System for Dynamic Multi-Layer 3D Gaussian Splatting Scenes. In *Proceedings of the 16th ACM Multimedia Systems Conference* (Stellenbosch, South Africa) *(MMSys '25)*. Association for Computing Machinery, New York, NY, USA, 136–147. doi:10.1145/3712676.3714445

[26] Babak Taraghi, Hermann Hellwagner, and Christian Timmerer. 2023. LLL-CAdViSE: Live Low-Latency Cloud-Based Adaptive Video Streaming Evaluation Framework. *IEEE Access* 11 (2023), 25723–25734. doi:10.1109/ACCESS.2023.3257099

[27] Jeroen van der Hooft, Stefano Petrangeli, Tim Wauters, Rafael Huysegems, Patrice Rondao Alface, Tom Bostoen, and Filip De Turck. 2016. HTTP/2-Based Adaptive Streaming of HEVC Video Over 4G/LTE Networks. *IEEE Communications Letters* 20, 11 (2016), 2177–2180. doi:10.1109/LCOMM.2016.2601087

[28] Victor Vasiliev, Bernard Aboba, and Jan-Ivar Hasselquist. 2023. WebTransport over HTTP/3. Working Draft, IETF. https://datatracker.ietf.org/doc/html/draft-ietf-webtrans-http3

[29] Linye Wei, Jiajun Tang, Fan Fei, Boxin Shi, Runsheng Wang, and Meng Li. 2025. No Redundancy, No Stall: Lightweight Streaming 3D Gaussian Splatting for Real-time Rendering. In *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–9. doi:10.1109/ICCAD66269.2025.11240950

[30] Jinbo Yan, Rui Peng, Zhiyan Wang, Luyang Tang, Jiayu Yang, Jie Liang, Jiahao Wu, and Ronggang Wang. 2025. Instant gaussian stream: Fast and generalizable streaming of dynamic scene reconstruction via gaussian splatting. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 16520–16531.

[31] Chenqi Zhang, Yu Feng, Jieru Zhao, Guangda Liu, Wenchao Ding, Chentao Wu, and Minyi Guo. 2025. STREAMINGGS: Voxel-Based Streaming 3D Gaussian Splatting with Memory Optimization and Architectural Support. arXiv:2506.09070 [cs.GR] https://arxiv.org/abs/2506.09070

[32] Mufeng Zhu, Mingju Liu, Cunxi Yu, Cheng-Hsin Hsu, and Yao Liu. 2025. SGSS: Streaming 6-DoF Navigation of Gaussian Splat Scenes. In *Proceedings of the 16th ACM Multimedia Systems Conference* (Stellenbosch, South Africa) *(MMSys '25)*. Association for Computing Machinery, New York, NY, USA, 46–56. doi:10.1145/3712676.3714437